```
});

const chat = await cohere.chat({.
  model: "command",
  message: "What is 7+7 and why?",
});
```

Uses the chat functionality, providing the parameters that it requires. Note that the parameters have different options than the OpenAI client.

Each provider has its own unique way of initializing the client and making API calls, leading to code duplication and increased maintenance. We need a better abstraction for using different clients—not to mention that we may need to create our own client from scratch that interfaces with a custom model in the future as well. Would it be ideal if we had an abstract way to utilize AI model providers with minimal configuration? To address this concern, the Vercel AI SDK offers a nice abstraction over the different AI providers using a language model specification.

3.3.2 Using the Vercel AI SDK's interoperability

The Vercel AI SDK addresses the challenges of working with multiple AI providers by offering a standardized language model specification. This specification acts as an abstraction layer, allowing developers to interact with various models using a unified interface, which is a powerful application of the abstract factory design pattern. This pattern decouples the client code from the specific model implementations, enabling you to switch providers with minimal changes. The following listing shows an example of how you can generate text using different providers and models with the Vercel AI SDK. I have omitted the part that configures the clients for simplicity.

Listing 3.10 Calling the `generateText` function with different models

```
import { generateText } from 'ai';
import { openai } from '@ai-sdk/openai';
import { anthropic } from '@ai-sdk/anthropic';

// OpenAI example
const openaiResponse = await generateText({.
  model: openai('gpt-3.5-turbo'),
  prompt: 'What is 7+7 and why? ',
});

// Anthropic example
const anthropicResponse = await generateText({.
  model: anthropic('claude-v1'),
  prompt: ''What is 7+7 and why?',
});
```

With the Vercel AI SDK, you can easily switch between providers and models by simply changing the model parameter in the `generateText` function. The SDK takes care of the underlying provider-specific details, providing a consistent and intuitive way to interact with different language models.